

Other information on SVM/Kernels (practical aspects and critical view)

Alessio Micheli

micheli@di.unipi.it



Dipartimento di Informatica
Università di Pisa - Italy

Summary of Main characteristics (pro and cons)



Dip. Informatica
University of Pisa

- Regularization embedded in the optimization problem (margin)
- Approximation of the theoretical structural risk minimization
(bound to vc-dim of hyperplane minimized with the solution [hard-margin, no K.])
- Convex problem: **training always finds a global minimum**
- Implicit feature transformation using kernels

But

- Must choose kernel and kernel parameters
- Batch algorithm
- Very large problems are (were) computationally intractable
 - Problems with more than 20,000 examples (*are*) *were* very difficult to solve exactly with standard approaches.
 - However, many solution are proposed nowadays (including *gradient descent* approaches (!))

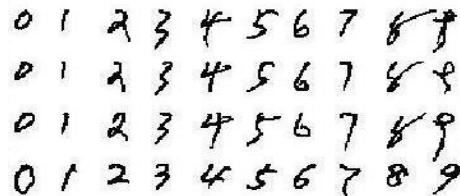
Advantages (**repetita**)



- Linear models (solver) with bound to the complexity in terms of margin (which is directly optimized)
 - Linear model (simplicity, compactness, ...)
- Rich set of non-linear decision functions in the input (sample) space via Kernels
 - we are still doing a linear separation, but in a different space
 - Efficient (by kernels) treatment of the linear basis expansion
 - Rich set “not directly” implies high VC-dim (which for SVM depends on margin, K. parameters, C, etc.)
- Kernel trick → Kernel Methods (also without SVM):
see later

SVM first (famous) application

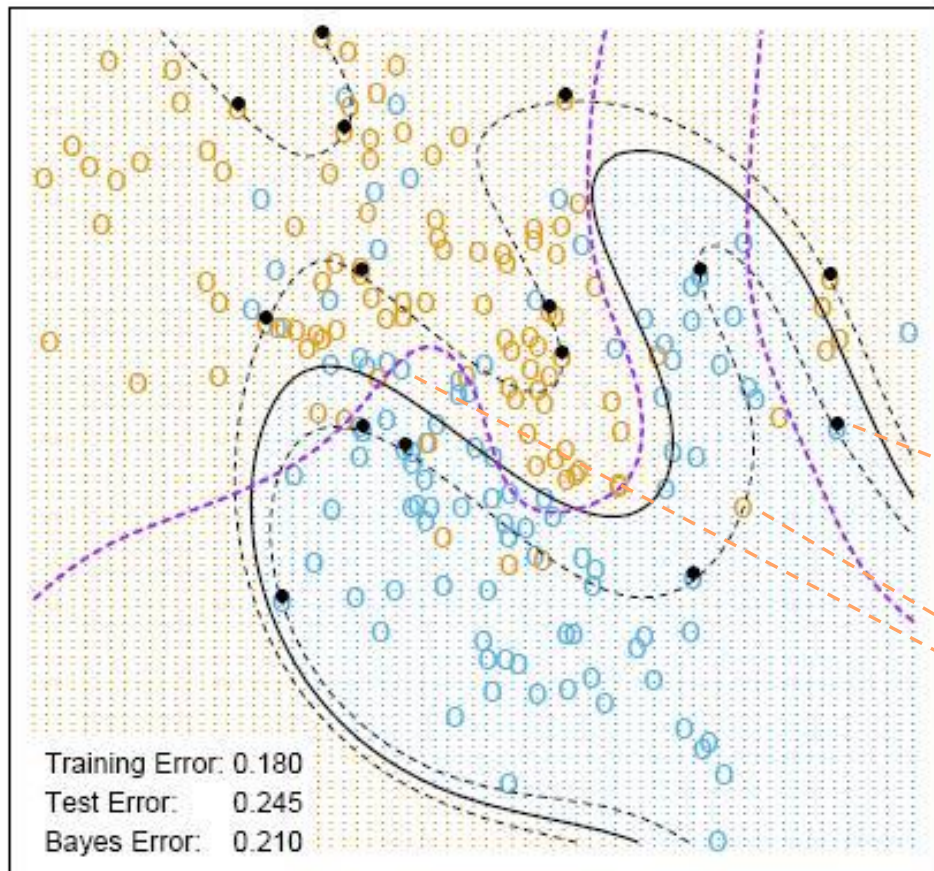
- The task we already know on handwritten digits (MNIST) represented an historical instance of successful applications for SVM



- E.g. 0.8% errors in ~1998 using a degree-9 polynomial SVM
- Which was noteworthy, comparable to LeNet, and obtained without years of developments on the architectures...
 - [the relevance of applications] Much more attention on SVM and SLT then followed.... (although related to 60'/70' studies)
 - [or] The relevance of good theory for successful applications

The mixture example: Polynomial Kernel results

SVM - Degree-4 Polynomial in Feature Space



C tuned to the best performance
 $C=1$ (Hastie et al.)

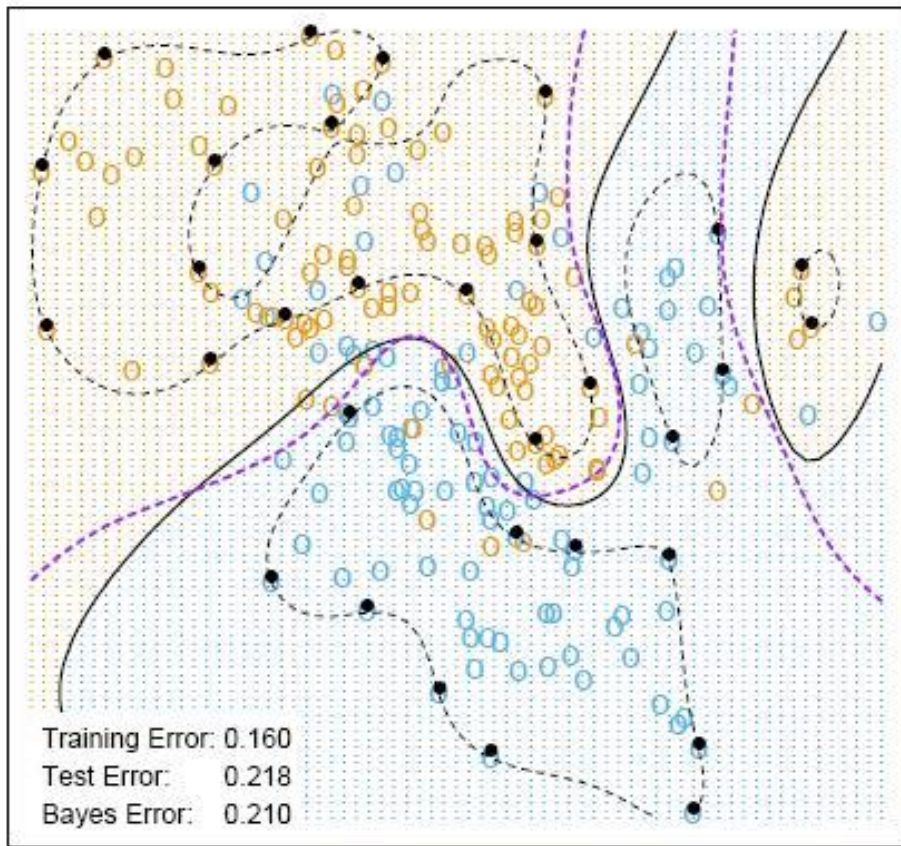
42% of data results to be
support vectors !

Support Vector: on the margin (slack=0)
or on the wrong side of the margin (slack > 0)

The mixture example

RBF Kernel results

SVM - Radial Kernel in Feature Space



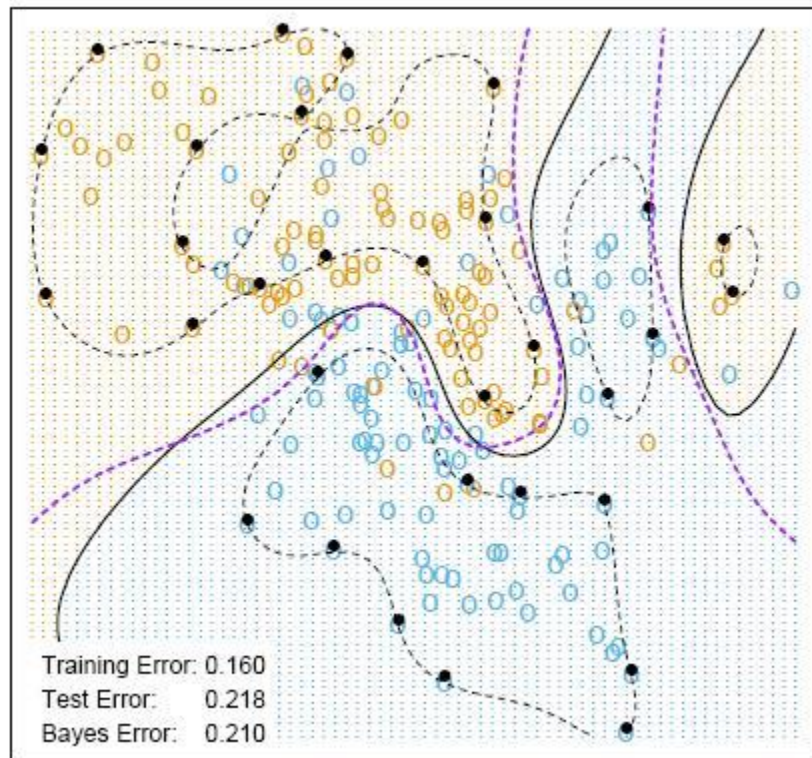
The radial basis kernel performs the best, as might be expected given the data arise from mixtures of Gaussians.

$\gamma = 1/(2\sigma^2) = 1$
45% of data are support vectors !

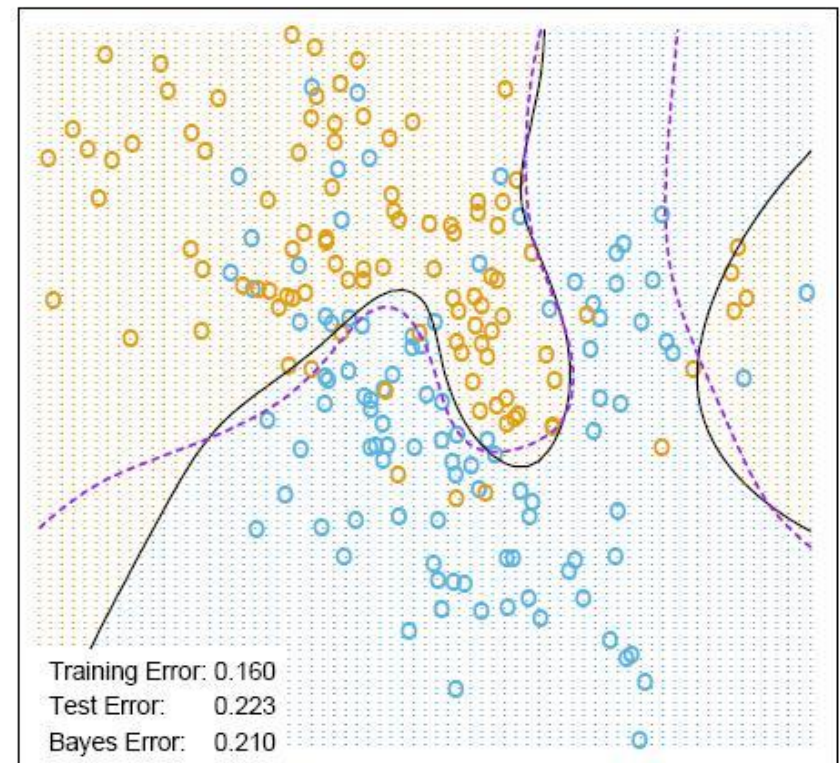
$$k(\mathbf{x}, \mathbf{x}_i) = e^{-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}_i\|^2}$$

Do you remember NN results?

SVM - Radial Kernel in Feature Space



Neural Network - 10 Units **Weight Decay=0.02**



SVM: practice of (I)

“Currently there exists no theory which *guarantees* that a given family of SVMs will have high accuracy on a given problem” [burges 98].

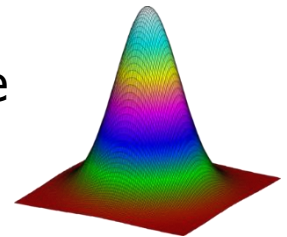
The nice property of hard-margin classifiers cannot be directly extended to soft-margin and kernels

(C parameter, Kernel can lead to infinite VC-dim of the SVM classifier)

E.g. :

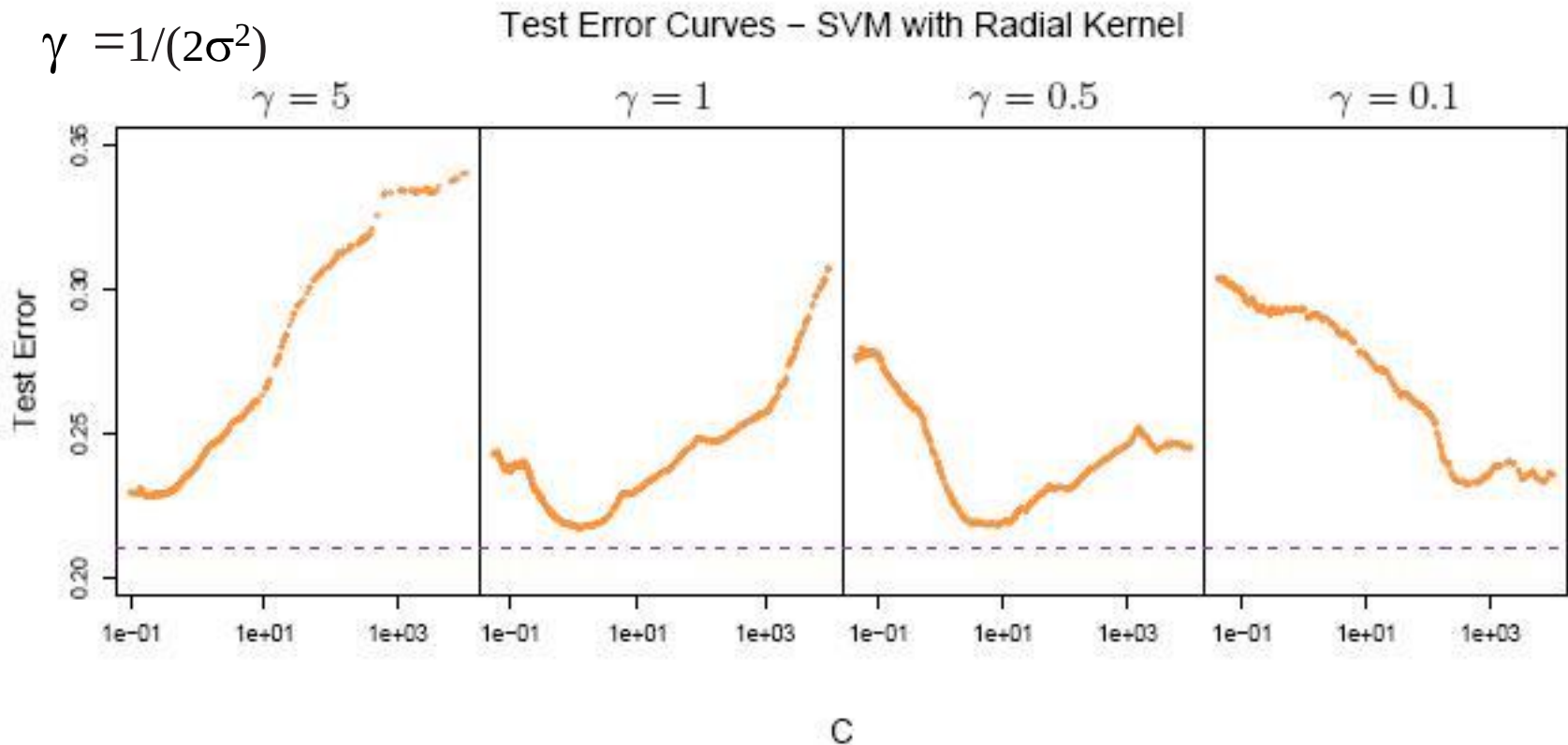
$$h(\mathbf{x}) = \text{sign}(\sum_{i=1}^N \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i))$$

- Gaussian RBF SVMs of sufficiently small width (σ) can classify an arbitrarily large number of training points correctly (like 1-NN only 1 SV point, the closet one to $\mathbf{x}$, will contribute to the solution), and thus it can have infinite VC dimension (unless you regularize the model)
- On the opposite with large width of the Gaussian all SV points are considered and you get a sort of "global average", low VC-dim



→ Controlling the width we can control the VC-dim

Critical role of Hyper-parameters on the performance on the performance



$\gamma = 5$ (small σ , narrow peaked Kernels) require lower C value (max regularization). Hence, the best value of C depends also on the kernel hyper-parameters !

Often selected using cross-validation: take care !

SVM: practice of (II)

- To apply the SVM to practical problems, one needs to choose the type of kernel (and its parameters) and the C value (also ϵ for regression).
- Rigorous selection requires an estimation of the complexity (VC-dim).
- Hyper-parameters affecting model complexity $\rightarrow$ hyper-parameters used for model selection
- In practice: again (carefully) empirical evaluation
 - Hyper-parameters fitting: tuning of hyper-parameters (C for soft-margin, γ for RBF K., etc.) can introduce **overfitting** (use a validation set to set hyper parameters and then a test set, or double-nested CV, etc....)

SVM: practice of (III)

- **Number of support vector**: assumed low in the discussion:
Not always true ! (see the examples at the beginning!)
- **Performance**: often good, *not always* the comparison is favorable w.r.t. other ML methods.
- E.g. Nowadays (after 2012/13) NN Deep model have shown to largely outperform previous records on image and speech recognition tasks!

SVM Folklore (and Exercise)

- Now you can answer by yourself to same typical erroneous issues on SVM (not *super* vector machine ;-)):
 1. SVM can be used with default hyper-parameters values
 2. I choose SVM because it does not fall into over-fitting
 3. SVM has not problems using high-input dimension (does it solve the curse of dimensionality?*)
 4. By SVM is like to find automatically the “number of units”
 5. ...

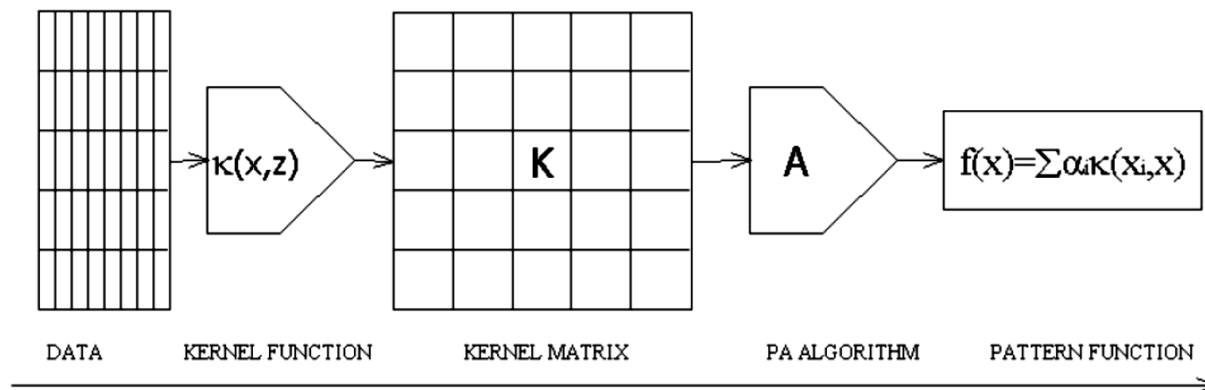
* Solution to 3: SVM does **not** solve curse of dimensionality of the input space !!
(as it proper deals with high dimensional in the feature space, not in the input space!)

Kernel Methods (I)

- Kernel trick $\rightarrow$ Kernel Methods (also without SVM)
- “Kernelization” of previous (traditional) approaches whenever you had a dot product inside your model or a similarity measure enabling them to operate in a new implicit high-dimensional space just specifying a *kernel* (and changing it)

Kernel Methods (II)

- Kernel trick $\rightarrow$ Kernel Methods (also without SVM)
 - Very efficient *fixed high-dim* **Linear basis expansion**
 - **Modular** choice of the basis functions by the kernel
 - **Unified theoretical framework** for general dot product functions



*K can change without changing
the learning machine!!!*

Kernel (III)

Moreover

- $K(s, t)$ can be related to a similarity measure comparing s and t

$$d_k(s, t) = \sqrt{k(s, s) - 2k(s, t) + k(t, t)}$$

- Similar objects (small distance) $\leftrightarrow$ high value of the kernel
- The idea underpinning the use of a kernel function is to focus on the representation and processing of **pairwise comparisons** rather than on the representation and processing of a single object.
 - For some domain it may be easier to compare two objects instead of defining some abstract space of features where to represent a single object.

Kernel (IV)

- An SVM is largely characterized by the **choice of its kernel**
 - Please, compare with the use of “adaptive” basis functions in NN : recall that the basis functions themselves were in NN adapted to data by fitting of w in ϕ (w of hidden units)
- Instead, the best choice of kernel for a given problem is still a research issue

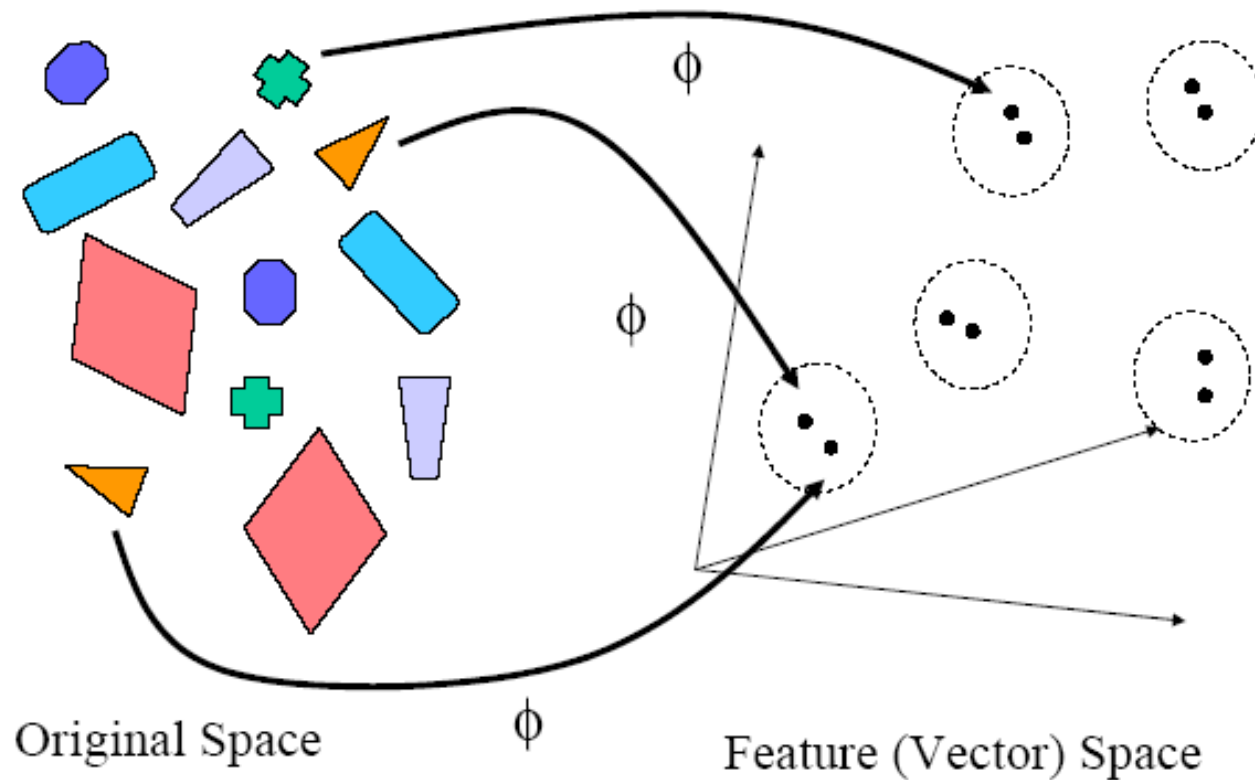
Kernel: Advanced topics (I)

- $K(s, t)$ can be related to a similarity measure comparing s and t

$$d_k(s, t) = \sqrt{k(s, s) - 2k(s, t) + k(t, t)}$$

- *Exercise:* Re-think it as for the distance-based methods (see k-nn):
 - when is it a good kernel? *
 - when is it a bad kernel? **

A good kernel for “objects”



Kernel: Advanced topics (II)

- **Design of new kernels:** Need to choose a “good” kernel function for the specific domain and it can be extended to generic object in inputs (not only vectors!) :
 - e.g. for strings/trees/graphs
 - ad hoc for the data domain and task (e.g. to encode similarity measure useful in NLP, bioinformatics, ...)*
 - efficiency (algorithm issues**),
 - adaptive kernels: a research issue
- * e.g. edit-distance kernel for bioinformatics or language suffix-tree for strings rep. → the comparison can be made in various efficient ways
Note that the background knowledge is sperated form the solver

ML Course structure

Where we are



Dip. Informatica
University of Pisa

